Stratégie de test

> Le module consacre sa séance 5 au test logiciel assisté par IA. Ce document doit donc tenir devant un enseignant qui vient d'enseigner le sujet.

1. Ce qu'on teste, et pourquoi pas le reste

Deux choses peuvent faire échouer ce projet le jour de la soutenance :

1. **une donnée personnelle qui part chez le fournisseur** — la règle centrale de l'énoncé ;
2. **un morceau de cours qui disparaît sans qu'on le sache** — le défaut que le découpage corrige.

Les deux sont **déterministes** et **hors réseau**. Ce sont donc elles qui sont testées en priorité, et c'est pourquoi la suite tourne en dix secondes sans aucune clé d'API.

Ce qu'on ne teste pas : la qualité des sorties du modèle. Un résumé n'est pas « juste » ou « faux » au sens d'un test. Cela se mesure sur de vrais appels, avec un corpus et des critères — c'est le banc d'essai (AA5), pas la suite de tests.

2. Chiffres

154 tests, 39 secondes, aucun appel réseau.

Fichier	Tests	Portée
`test_redact.py`	23	anonymisation : détection, faux positifs, secrets, restauration
`test_segmentation.py`	19	découpage, recouvrement, sélection BM25, répartition
`test_api.py`	42	bout en bout : HTTP → routes → pipeline → base
`test_import.py`	16	import PDF, `.docx`, `.pptx` : ordre, tableaux, notes, refus
`test_examen.py`	24	examen blanc : secret des réponses, coût nul, double remise
`test_recherche.py`	17	recherche et tuteur transversaux : coût, ancrage, cloisonnement
`test_consommation.py`	13	tableau de coûts : exactitude des comptes, cloisonnement

Une famille de tests porte sur un coût, pas sur un résultat. Trois des fonctionnalités récentes affirment ne rien coûter — composer un examen blanc, chercher dans ses cours, consulter sa consommation. Ces affirmations sont dans le rapport et seront dites en soutenance : elles comptent les appels à la doublure avant et après, plutôt que d'être simplement crues.

3. Les trois niveaux

Unitaire — `test_redact.py` , `test_segmentation.py`

Fonctions pures, sans base ni réseau. Les cas difficiles sont **volontaires** : un jeu de tests qui ne contient que des cas faciles ne prouve rien.

Exemples de cas réellement discriminants :

- un numéro tunisien `+216 55 123 456` (groupes 2-3-3) **et** un numéro français `06 12 34 56 78` (groupes de 2) ;
- une plage d'années `2022-2026` qui doit **rester** ;
- un bloc de 5 000 caractères sans la moindre ponctuation ;
- une question dont aucun mot n'apparaît dans le cours.

Bout en bout — `test_api.py`

HTTP → routes → authentification → anonymisation → découpage → validation → persistance. **Seules deux pièces sont doublées** :

Pièce réelle	Doublure	Pourquoi
PostgreSQL	SQLite en mémoire	même schéma, créé depuis les mêmes modèles
Gemini	fonction qui rend du JSON conforme	un test qui dépend du réseau n'est pas un test

Tout le reste est le code de production. C'est ce qui rend ces tests utiles : ils échouent si l'une de ces pièces se casse.

La doublure du modèle **compte les appels**. C'est elle qui permet de vérifier qu'un cours long déclenche bien *n+1* appels, et qu'un cours déjà résumé n'en déclenche aucun.

Manuel — navigateur

Vérifié le 12 octobre : inscription, connexion, création de cours, liste, fiche à onglets, onglet conservé dans l'URL, et affichage de `LLM_KEY_MISSING` sans clé.

4. Les tests qui portent une exigence

Test	Exigence
`test_aucune_donnee_personnelle_ne_part`	« jamais de données personnelles brutes envoyées au LLM »
`test_les_bonnes_reponses_ne_descendent_pas`	l'auto-évaluation a un sens
`test_sortie_non_conforme_refusee_apres_une_relance`	« sortie validée avant affichage »
`test_absence_de_clef`	« gestion des erreurs d'appel API »
`test_la_fin_du_cours_est_atteinte`	le cours n'est pas tronqué
`test_aucun_chapitre_n_est_perdu`	rien ne disparaît **au milieu** non plus
`test_un_etudiant_ne_voit_pas_le_cours_d_un_autre`	la propriété est vérifiée côté serveur
`test_un_support_importe_passe_par_le_masquage`	le nouveau chemin d'import n'ouvre pas de brèche dans l'anonymisation
`test_le_contenu_des_tableaux_est_extrait`	un support Word n'est pas lu à moitié
`test_les_notes_de_l_orateur_sont_incluses`	un diaporama n'est pas lu à 69 %

Le second test de découpage mérite une note : atteindre la fin ne suffit pas. Un découpage qui saute un paragraphe serait aussi grave que l'ancienne troncature, et bien plus difficile à repérer. Le test vérifie donc que les quarante chapitres d'un cours synthétique sont tous présents.

4 bis. Comment nous savons que ces tests protègent vraiment

Les seize tests d'import sont passés **du premier coup**. C'est précisément le moment de se méfier : un test écrit en même temps que le code qu'il vérifie peut très bien ne vérifier que lui-même.

Nous avons donc cassé volontairement chacune des trois protections et relancé :

Protection retirée	Tests tombés
Le tri des formes par position	1
Le parcours du document Word dans l'ordre	2
Le filtre du numéro de diapositive	**0**
La fenêtre d'extrait centrée sur le mot cherché	**0**
Le secret des réponses de l'examen	8
Le refus d'une seconde remise de copie	1
Le cloisonnement du tableau de consommation	1

Le troisième cherchait la sous-chaîne `" 7 "`. Or le numéro de page est en bas de la diapositive, donc trié en dernier, donc sans retour à la ligne après lui : l'assertion ne pouvait pas échouer. Réécrite sur les **lignes** du

texte extrait, elle tombe bien quand le filtre est retiré.

Un test vert qui ne protège rien est pire qu'un test absent : il donne confiance. La parade coûte une minute et devrait être systématique sur tout test écrit en même temps que son code — **casser exprès ce qu'il protège.**

5. Les tests qui documentent une limite

Un test peut servir à **constater** plutôt qu'à exiger. Deux exemples :

- `test_limite_connue_le_nom_d_un_tiers_n_est_pas_detecte` — affirme qu'un patronyme **n'est pas** masqué. C'est le constat, pas le souhait. Le jour où une reconnaissance d'entités sera ajoutée, ce test échouera, et c'est exactement ce qu'on veut.
- `test_ne_masque_pas_une_suite_d_annees` — protège contre un sur-masquage qui détruirait du contenu pédagogique.

Mieux vaut un défaut écrit dans la suite de tests qu'un défaut découvert en soutenance.

6. Ce que les tests ont trouvé

Trois défauts réels, **aucun trouvé par relecture** :

Date	Défaut	Trouvé par
12 oct.	`+216 55 123 456` partait en clair : le motif n'acceptait que des groupes de deux chiffres	test de bout en bout
12 oct.	la liste des cours renvoyait 500 : colonnes de sous-requête mal nommées	test de bout en bout
12 oct.	la vue de transparence affichait toujours les mêmes types masqués	appel réel

Et un **faux positif de ma part** : j'ai d'abord cru à un défaut du tuteur, qui répondait « hors du cours ». Vérification faite, le mot cherché n'existait pas dans le document — le modèle avait raison. C'est aussi un enseignement : un test mal écrit accuse le code à tort.

7. Commandes

```
# Tests
cd server && .venv/Scripts/python.exe -m pytest tests -q

# Un seul fichier, en détail
.venv/Scripts/python.exe -m pytest tests/test_segmentation.py -v

# Front
cd client && npx tsc --noEmit && npm run build

# Migration : produit le SQL sans se connecter
cd server && python -m alembic upgrade head --sql
```

8. Intégration continue

`.github/workflows/ci.yml`, à chaque poussée et chaque demande de fusion :

1. installation des dépendances Python ;
2. `pytest` ;
3. **vérification du câblage** : l'application se construit entièrement — routes, dépendances, modèles de réponse — sans base ni clé ;
4. **migration hors ligne** : le SQL doit être valide ;
5. compilation et construction du front.

Aucun appel réel au fournisseur en CI : lent, coûteux, instable.

9. Couverture

Pas de seuil chiffré, et c'est délibéré. Un pourcentage se satisfait en testant des accesseurs. Ce qui compte ici :

- `redact.py` et `segmentation.py` : couverts branche par branche, y compris les cas limites ;
- les trois modules IA : couverts de bout en bout, cours court **et** cours long ;
- l'authentification et la propriété : couvertes ;
- le front : compilation et construction vérifiées, pas de tests unitaires. Limite assumée — le temps disponible a été mis sur le back, où vivent les règles.

10. Définition du « terminé »

Une fonctionnalité est terminée quand :

- [ ] les tests passent ;
- [ ] le front compile et se construit ;
- [ ] la migration produit du SQL valide, s'il y a eu un changement de schéma ;
- [ ] **la fonctionnalité a été exécutée**, pas seulement relue ;
- [ ] la documentation concernée est à jour.

L'avant-dernier point n'est pas une formalité : sur ce projet, **toutes** les erreurs réelles ont été trouvées à l'exécution.